CN5006 Lab Tutorial: React Programming using VSC

Prepared by: Dr.Nadeem Qazi

This tutorial introduces you basic of React programming and how to create to functional component to create user interface.

Note: A video tutorial of this lab is also present on the following link on the moodle.

https://moodle.uel.ac.uk/pluginfile.php/5155142/mod_label/intro/Meeting%20with%20Nadeem%20Qazi%201700-20261109_170512-Meeting%20Recording.mp4

At the end of this Lab tutorial you will be able to do following:

- How to create a React Application
- Creating the Functional Component using properties
- Adding Click Event for Functional Components

How to create React App

1. If you have not done it before, You must installed Node js on your laptop.
2. Create a new directory React work or with any othername in your laptop

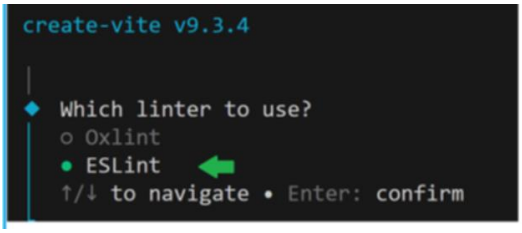
Use VSC terminal to run npx command.

Open Visual Studio and open the folder myReact that you just created by opening the terminal window and using the cd command followed by the path. In my case, it is cd ***C:\Users\nqazi\NQ\ReactWork, you should check your folder.***

In terminal window write following :

npm create vite@latest my-react-app -- --template react

Note you can myapp is the name of React application, you can give any name. press enter and select ESLint as shown in the screen shot



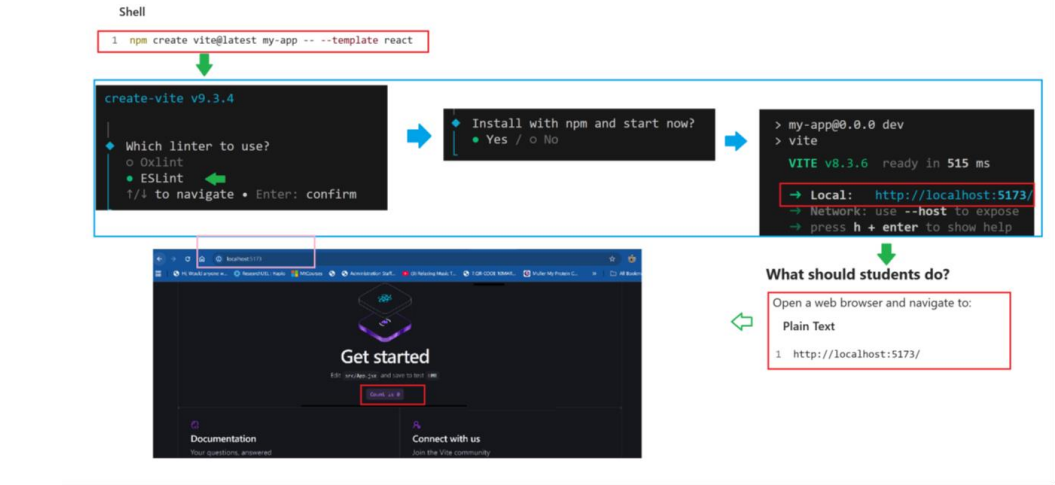
Complete steps are shown in the figure below:

How to create a React Application using visual Studio Code

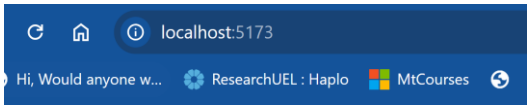
- Install the latest version of Node.js, which includesnpm.
- Open Visual Studio Code and launch a terminal.
- Create a new React application using Vite:

Shell

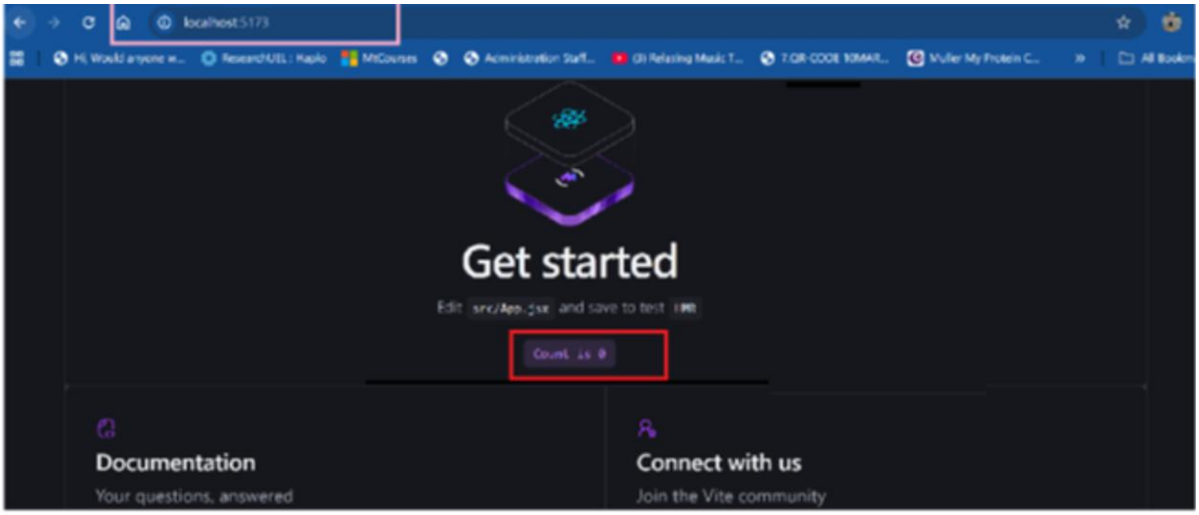
```
1  npm create vite@latest my-app -- --template react
```



Once you see Local:http://localhost:5173, Open the browser and write following :



Your application will start running at port 5173 and you will see the following browser screen.its counting React App. Click button to count.

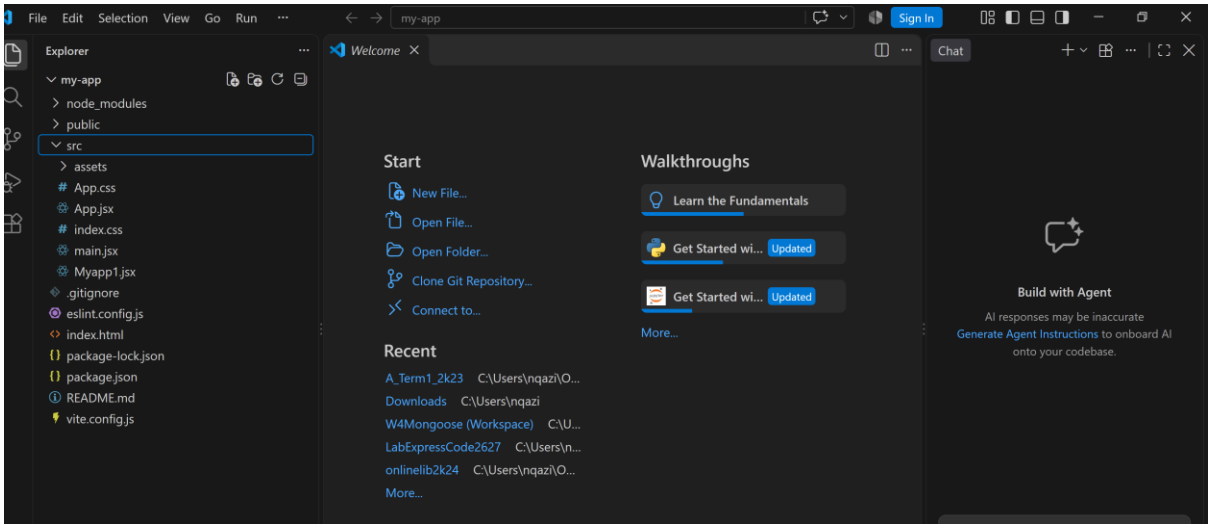


Click button to see its counting.
Now switch back to the command Terminal again and presee Ctrl-C the application will stop running.

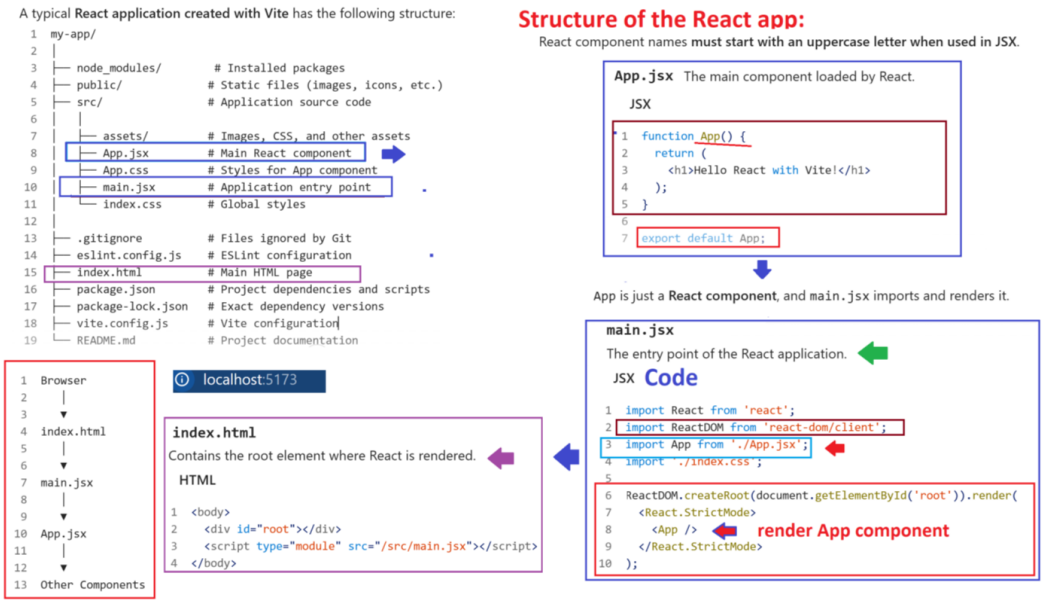
Structure of the React app:

- Navigate to the folder where you created the React application.
- Locate the directory named myReact.
- Inside myReact, find the folder named myapp (or the specific name you used on your system).
- Open this folder in Visual Studio to begin working with your React application,if it is not opened before.

Look at the structure of the React application created for you by the React library: You will see the following screen on your Visual studio



React Application Structure



1. Public Directory
    a. Index.html: which is displayed as web page in the browser
2. Src Directory
    a. App.jsx: is the JS file that contains component
    b. main.jsx : is the main file that produce the index.html which runs in browser

Note: Most of the Time you will be editing or creating app.jx and using it in main.jsx to code any new feature in your React Application
Also, observe there are two CSS files:
i) app.css file: for setting css for your app.js
ii) Index.css : for the overall css of your application

You can run the application by typing npm run dev on the terminal window

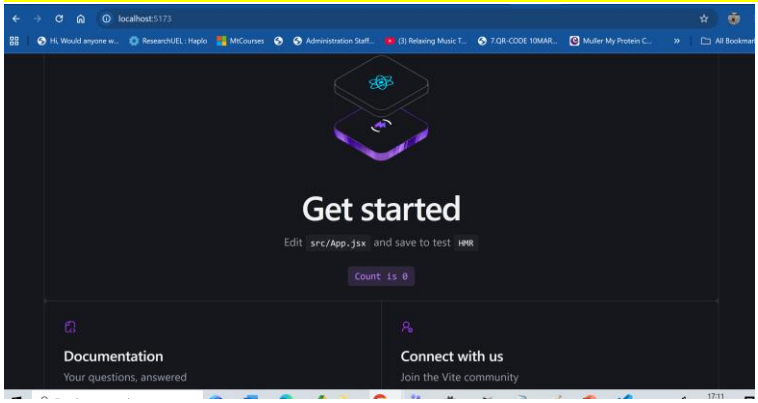


You will see following in the console window

```
VITE v8.3.6  ready in 874 ms

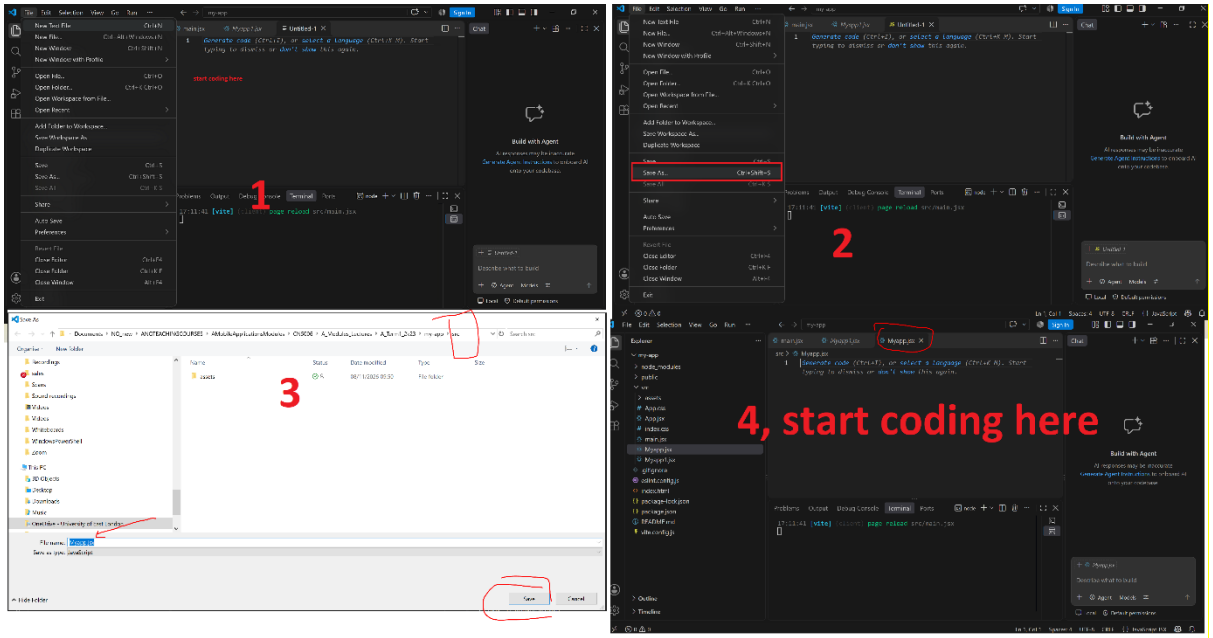
➜ Local:   http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
```

Test it , opening a browser window and in location bar type http://localhost:5173



Task1 : we will create a functional component with properties and use it in main.jsx, follow these steps

Create a Myapp.jsx file using the sequence below: you need to click file, create new file and save it as MyApp.jsx as shown below.

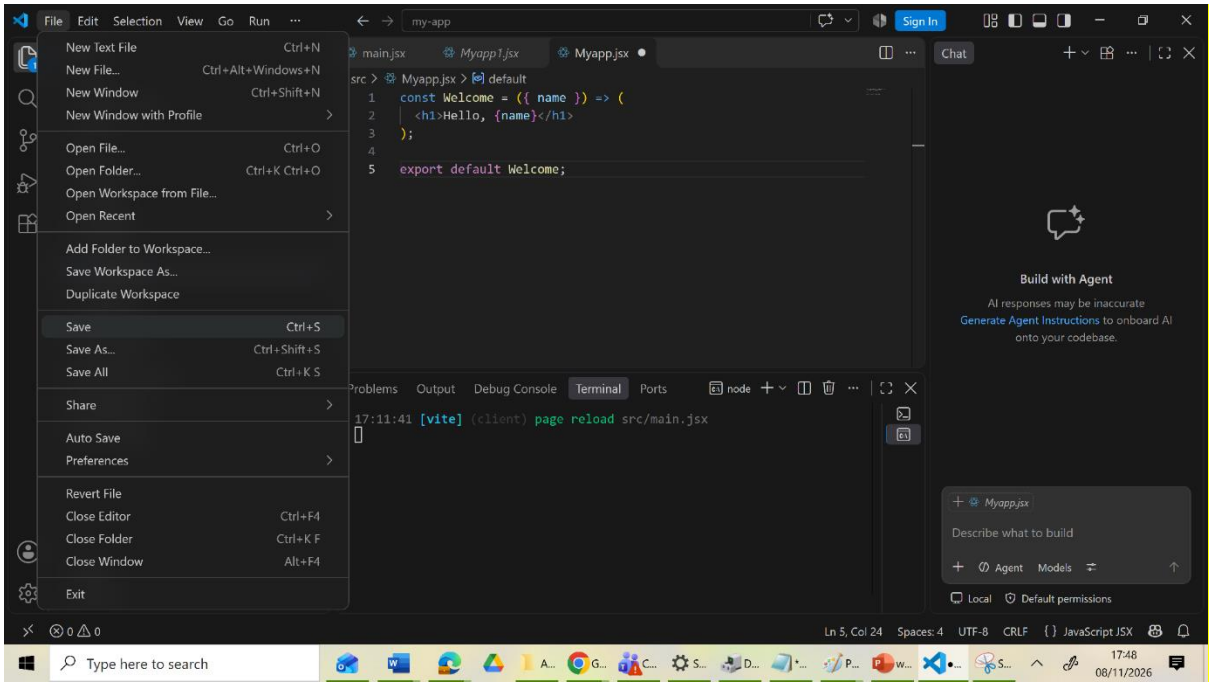


Copy following code into MyApp.jsx and save it

```
const Welcome = ({ name }) => (
```

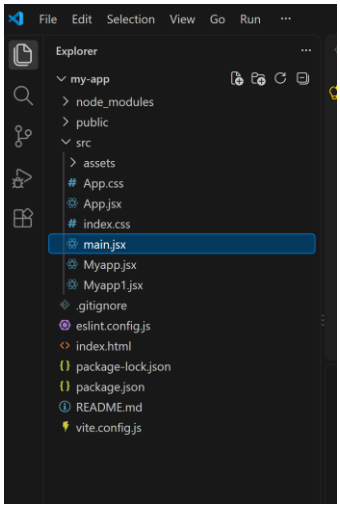
```
    <h1>Hello, {name}</h1>
  );
```

```
export default Welcome;
```



Observe functional Component starts with Capital letter

Now click main.jsx file



Write the following code exactly as shown in the figure below. Also, comment out the App component using /* <App /> */ as shown. Note that the App component is commented out and will not be rendered. You can simply copy and paste the code into main.jsx.

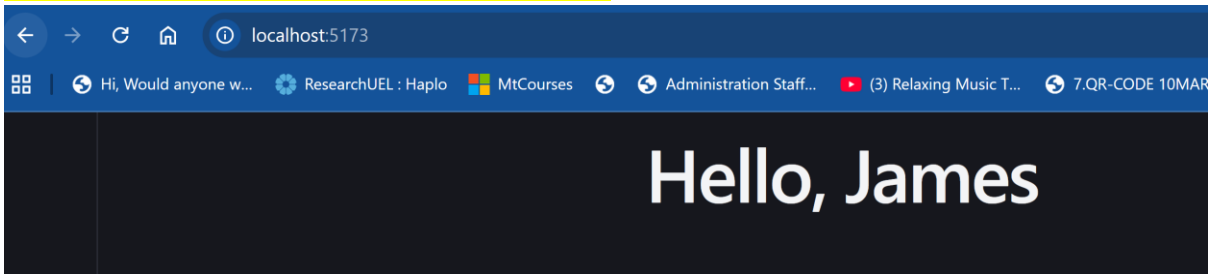
Main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
//import App from './App.jsx'
import Welcome from './Myapp.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>

    {/*<App/>*/}
    <Welcome name= "James"/>
  </StrictMode>
)
```

Save it. You will see following. You just have created one functional component Welcome that takes one property name. and in Main.jsx you pass “James”, you can use other name to see the changes.



How does a React application works

1. **React application display index.html which is created by index.js.**

2. **What index.js does:**

3. The content of the index.html is created by index.js.
4. Main.jsx uses a function called as render() to render any element component or any jsx code. These elements, components or JSX code, however, may also be defined in some other js files such as app.js.
5. App.js or any other js file that holds the definition of your component must be imported in main.js as shown in the figure

Task 2: Functional Components Using Properties and HTML Elements

In this task, we will create a React functional component that displays information about a product. The component will receive properties such as the **product name** and **price** and display them using HTML elements such as headings, paragraphs, labels, input fields, and buttons.

The component will include an input field that allows the user to enter a **VAT rate (%)** and a **Calculate VAT** button. When the user clicks the button, the application will calculate the product price including the VAT percentage entered by the user and display the result on the webpage.

For example, if the product is a **Laptop** priced at **£800** and the user enters a VAT rate of **20%**, clicking the button will display:

Price including VAT: £960

Like before create a separate Prodcut.jsx file using visual Studio Code and write the following code in product.jsx., you can simply copy the code from here.

```
const Product = ({ productName, price }) => {

  const calculateVAT = () => {

    const vatRate = document.getElementById("vatRate").value;

    const priceWithVAT =
      Number(price) + (Number(price) * Number(vatRate) / 100);

    document.getElementById("vatMessage").innerHTML =
      `<b style="color:green">Price including VAT: £${priceWithVAT}</b>`;
  };

  return (
    <div>
      <h2>{productName}</h2>

      <p>Price: £{price}</p>
```

```
      <label>Enter VAT Rate (%): </label>

      <input
        type="number"
        id="vatRate"
      />

      <br /><br />

      <button onClick={calculateVAT}>
        Calculate VAT
      </button>

      <p id="vatMessage"></p>

    </div>
  );
};

export default Product;
```

What this code Does:

- **The line `const Product = ({ productName, price }) => { ... }` defines a functional component that receives two props, `productName` and `price`, from the parent component and displays their values on the webpage.**

- **The component contains a function named `calculateVAT()`. This function is responsible for reading the VAT rate entered by the user, performing the VAT calculation, and displaying the result.**

- **The button uses the code `onClick={calculateVAT}`. This means that whenever the user clicks the `Calculate VAT` button, React calls the `calculateVAT()` function automatically.**

- **Inside the `calculateVAT()` function, the statement `document.getElementById("vatRate").value` retrieves the VAT percentage entered into the input field, and the calculation `Number(price) + (Number(price) * Number(vatRate) / 100)` computes the final price including VAT.**

- **The statement `document.getElementById("vatMessage").innerHTML = ...` updates the paragraph element with the id `vatMessage` and displays the calculated price in bold green text below the button.**

Then add following line to include product component in the main.jsx

```
import Product from './Product.jsx'
```

Then add following line into main.jsx to call product component

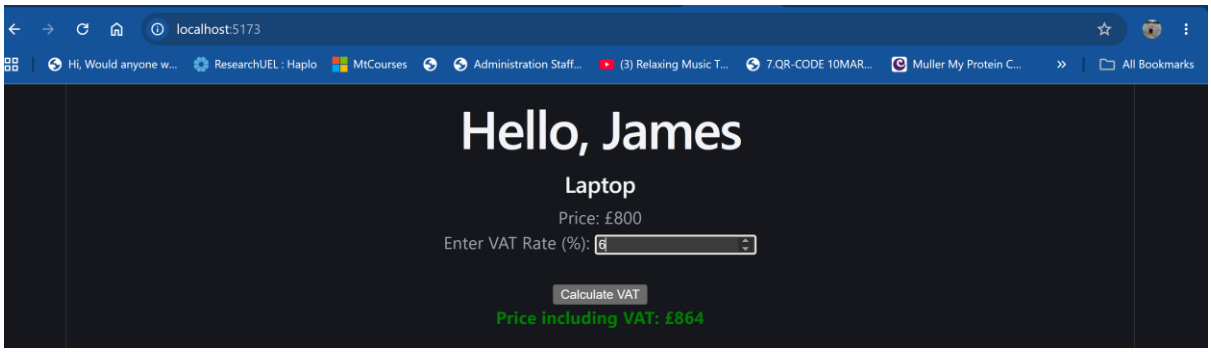
```jsx
<Product productName="Laptop" price={800} />
```

Your full main.jsx code look like this:

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
//import App from './App.jsx'
import Welcome from './Myapp.jsx'
import Product from './Product.jsx'
createRoot(document.getElementById('root')).render(
  <StrictMode>

    {/*<App/>*/}
    <Welcome name= "James"/>
    <Product productName="Laptop" price={800} />
  </StrictMode>
)
```

Save the file, you will see following localhost:5731



Task for the portfolio:

Portfolio Task: Discount Calculator Component

Task Description

Create a React functional component called **DiscountCalculator.jsx**. The component should be similar to the **Product.jsx** example covered in the lab but should calculate a **discounted price** instead of VAT.

The component must:

- Receive **productName** and **price** as props.
- Display the product name and price using appropriate HTML elements.
- Include a text box where the user can enter a **discount percentage**.
- Include a button labelled **Calculate Discount**.
- When the button is clicked, calculate the final price after applying the discount.
- Display the calculated discounted price below the button in a different colour and bold text.

Submission Requirements

Submit the following:

1. **DiscountCalculator.jsx** containing the complete React functional component.
2. A screenshot showing:
    - The component displaying the product name and price.
    - A discount percentage entered by the user.
    - The calculated discounted price displayed after clicking the button.
3. A brief explanation (100-150 words) describing:
    - The purpose of the component.
    - The props used in the component.
    - How the button triggers the discount calculation.
    - How the result is displayed on the webpage.